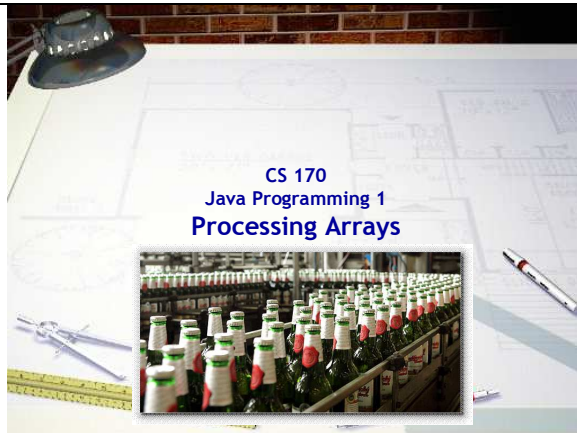




Slide 1

CS 170

Java Programming 1

Processing Arrays

Duration: 00:00:43

Advance mode: Auto

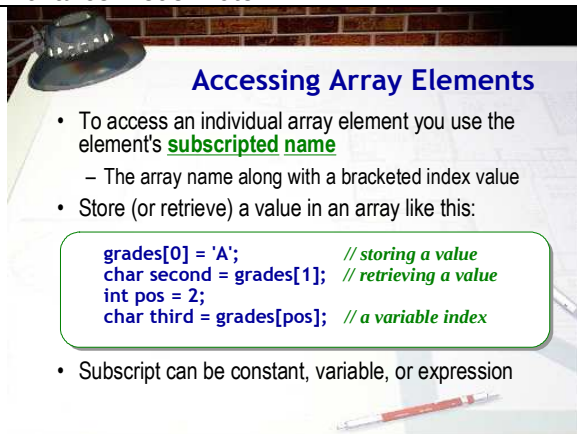
Notes:

Hi there. Welcome to the CS 170, Java Programming I lecture on Processing Arrays.

Now that you know how to create arrays, it's time to put them to work. We don't want all of those variables just sitting around collecting dust.

In this lecture, you'll first learn about storing and retrieving values in individual elements. Then, you'll learn how to write methods that take arrays as parameters and how to write methods that produce array values.

As you work through this lecture, we'll continue to add to code to the ArrayTest program and its run() method. You should, though, start a new section in your IC document for this lecture's exercises.



Slide 2

Accessing Array Elements

Duration: 00:01:10

Advance mode: Auto

Notes:

To access an individual array element, you use the name of the array and add the subscript of the element you want to access in brackets following the name. Using this "subscripted" name, you can store new values in individual elements, or retrieve values from individual elements in the array.

For example, to store a new grade into the first element of grades, you just do this:

```
grades[0] = 'A';
```

using the subscripted name on the left side of an assignment expression. In the same way, you can retrieve a value simply by using the subscripted variable name on the right-side of an assignment expression, like this:

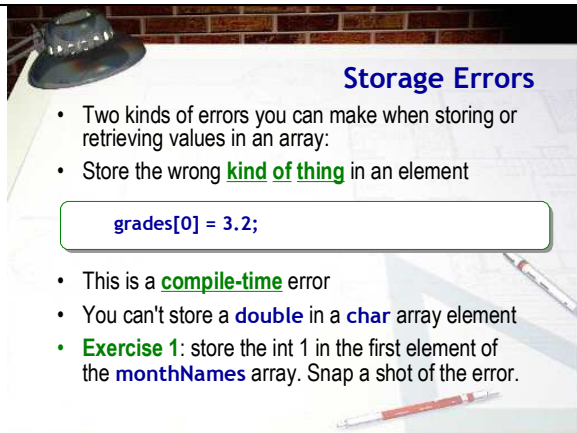
```
char second = grades[1];
```

You don't have to use a literal value like 1 or 2 for the subscript index. You can use a variable like this

```
int pos = 2;
char third = grades[pos];
```

You can also use an expression as long as the result is an integer value.

Although the type of the variable grades is char[] (array of char), the type of the individual elements (grades[1]) is plain-old char. You treat each of the individual elements just as if it were a simple char variable.



Storage Errors

- Two kinds of errors you can make when storing or retrieving values in an array:
- Store the wrong **kind of thing** in an element

```
grades[0] = 3.2;
```

- This is a **compile-time** error
- You can't store a **double** in a **char** array element
- Exercise 1:** store the int 1 in the first element of the **monthNames** array. Snap a shot of the error.

Slide 3

Storage Errors

Duration: 00:01:07

Advance mode: Auto

Notes:

There are two kinds of errors you can make when storing or retrieving elements from an array.

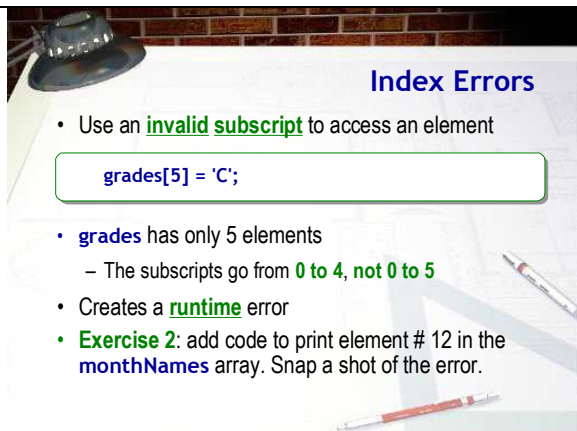
When storing elements, you might try to put the wrong kind of thing into an element. You have to remember to store only compatible values (that is, values where assignment to simple variables of the same type would be acceptable). If you try this, for instance:

```
grades[0] = 3.2;
```

you'll get a compiler error because you can't store a double value, (3.2), in a char variable. It's the wrong kind of thing.

Let's make sure that's completely clear.

Start a new section in your IC document and for Exercise 1, try to assign the integer value 1 to the first element in the monthNames array. Save your code and switch to the problems tab. Locate the error and snap a screen-shot of it. Remember the error message so that when you encounter it in your own programs, you'll remember what the cause is. Once you've shot your picture, comment out the line to remove the error.



Index Errors

- Use an **invalid subscript** to access an element

```
grades[5] = 'C';
```

- grades** has only 5 elements
 - The subscripts go from **0 to 4**, not 0 to 5
- Creates a **runtime** error
- Exercise 2:** add code to print element # 12 in the **monthNames** array. Snap a shot of the error.

Slide 4

Index Errors

Duration: 00:01:10

Advance mode: Auto

Notes:

A more common mistake is to create a runtime error by trying to either store or retrieve values from an element outside the bounds of the array. The most common cause of this error is forgetting that a five-element array, like grades, has subscripts that range from zero to four. The expression

```
grades[5] = 'C';
```

will trigger a runtime error, (an `ArrayIndexOutOfBoundsException` exception). You'll learn how to deal with this error later in the course when you look at exceptions. For right now, though, just remember that every subscript must fall in the range 0 to size-1 where size is the number of elements in the array.

For Exercise 2 let's try writing some code that illustrates this error. Add a statement to print the value of `monthNames[12]` using `System.out.println`. Your code should compile without error. Now, go ahead and run the program and you'll see the runtime error. Snap me a screenshot of the error.

Array Length

- Every array has a public **field**, **length**, that stores the number of elements in the array
- When using the **length** field, **remember** :
 - Strings have a **length()** **method**, arrays have a **field**

```
String s = "Abara";
int lenS = s.length();
int lenA = grades.length;
```

- The last element in **any** array is at index **a.length-1**

Slide 5

Array Length

Duration: 00:00:53

Advance mode: Auto

Notes:

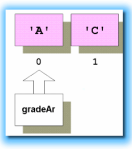
You don't have to search back through your code and try to remember how big each array is, though. Every array variable has a public field named **length** that you can use to discover how many elements are in the array.

There are two things you should remember when using the **length** field:

- In an array, **length** is a field; in a String, **length()** is a method. It's easy to get the two of these confused. String `s = "Abara";`
`int lenS = s.length();`
`int lenA = grades.length;`
- The **length** field stores the number of elements in the array, not the highest subscript number, which is always **length-1**.

Variables vs. Arrays

- Variables and **actual** array are different
 - Array variable **gradeAR** **refers to** the actual elements containing 'A' and 'C'
 - Actual elements located somewhere on the runtime heap
- Using **gradeAR** like this modifies the elements of the actual array, not variable **gradeAR**



```
23:
24: // give second student a B
25: gradeAR[1] = 'B';
26:
27:
```

Slide 6

Variables vs. Arrays

Duration: 00:00:42

Advance mode: Auto

Notes:

By now you should understand that an array variable and the actual array where the array elements are stored, are two different things, as this picture fragment, from an earlier lecture, shows. In this example, the array variable, **gradeAR**, refers to the actual array elements 'A' and 'C', located "somewhere else" in memory in an area called the "runtime heap".

When you use assignment with **gradeAR** as part of a subscripted variable name, like this:
 you are not modifying the variable **gradeAR**, but are changing the value in the second element of the actual array that **gradeAR** "points to".

Arrays as Parameters

- Cannot** pass actual arrays as parameters to methods
 - Can pass individual elements or array variables
 - Effect is the same as passing the array
- Use **type[]** as the formal parameter like this:

```
/* the array will contain at least one element.
*/
public boolean firstLast6(int[] nums)
{
    int last = nums.length - 1;
    return nums[0] == 6 || nums[last] == 6;
}
```

Slide 7

Arrays as Parameters

Duration: 00:00:54

Advance mode: Auto

Notes:

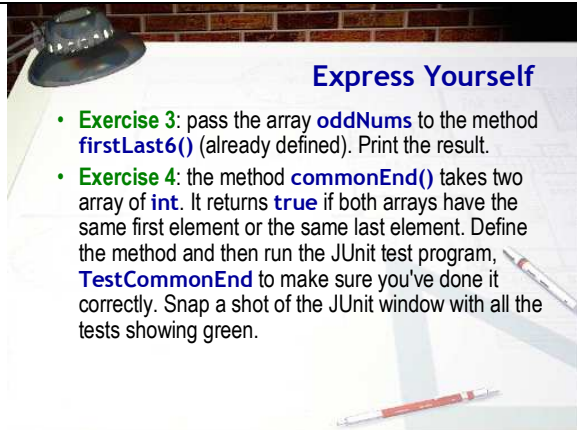
Although you cannot use an actual array as an parameter to a method, you can pass array variables as parameters. You can also pass individual elements, or write methods that return array variables.

Lets see how that works.

To pass an array variable to a method, you first have to know how to write the formal parameter.

Suppose, for instance that you write a method that returns true if either the first or last element in an int array was the value 6. The method header for the array parameter would not be an int, but an array of int, written as **int[]**. Notice that you can pass any size array to this method because it uses the array's **length** field to determine which element is the last element in the array when it comes time to do the

comparison.



Express Yourself

- **Exercise 3:** pass the array `oddNums` to the method `firstLast6()` (already defined). Print the result.
- **Exercise 4:** the method `commonEnd()` takes two array of `int`. It returns `true` if both arrays have the same first element or the same last element. Define the method and then run the JUnit test program, `TestCommonEnd` to make sure you've done it correctly. Snap a shot of the JUnit window with all the tests showing green.

Slide 8

Express Yourself

Duration: 00:01:54

Advance mode: Auto

Notes:

On the previous slide, you saw the method `firstLast6()`, which is actually already defined in the `ArrayTest` class that you've been working with. Let's do two things:

- see if you can pass an array to a method
- see if you understand how to define a method with array parameters

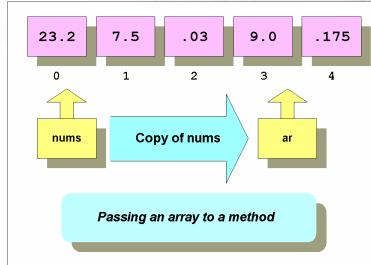
Let's start by passing an array. You've already defined an `int` array named `oddNums` in the previous lecture. Inside your `run()` method, pass `oddNums` to the `firstLast6()` method and print the result that you get back. (It should print `false` because neither the first, nor the last element in the array is a six.) Shoot me two screenshots for Exercise 3: one of your code in `run()` and one of the printout when you run `ArrayTest`.

For Exercise 4 I want you to define a method, inside the `ArrayTest` class, that takes two parameters; both parameters should be `int` arrays. The method will be named `commonEnd()` and it is a boolean method. It should return `true` if both arrays have the same first element or the same last element. Note that both arrays will not necessarily be the same length, so you'll have to calculate the position of the last element for each one independently before comparing the values.

Once you've defined your method and made sure it compiles, run the JUnit test program I've provided in this week's project, named `TestCommonEnd`. If you've done it correctly, all of the tests should show green. Shoot me a screenshot showing your code plus the JUnit test window.

Passing Arrays to Methods

- What happens when you pass an array variable to a method? (Assume an array variable named **nums**)



Notes:

Let's assume that you have an array variable named **nums** that refers to an array of doubles as shown in this slide. Now, suppose you pass that array variable to a method that has a formal parameter named **ar**. When you do this, a copy of the variable **nums** is made and then placed in the variable **ar**. This is the same as with any variable. All method parameters are passed by value in Java--a copy of the actual parameter is made for use inside the method.

However, with an array, the effect might not be what you expect. Let's see why.

Slide 9

Passing Arrays to Methods

Duration: 00:00:43

Advance mode: Auto

Modifying Array Elements

- When passed, the **reference** to the array is sent
 - Formal parameter (**ar**) and actual argument (**nums**) **both refer to the same array**
 - This means elements **MAY** be modified inside methods

```
void doubleIt(double ar[])  
{  
    for (int i = 0; i < ar.length; i++) ar[i] *= 2;  
}
```

- Array elements passed **cannot** be modified

Notes:

Even though a copy of the variable **nums** is made and placed in the formal parameter **ar**, because both **nums** and **ar** refer to the same actual array elements, any changes made to the elements of the array **ar** inside the method apply to the elements of the array **nums** as well. To reiterate, both **nums** and **ar** refer to the same actual array, even though **ar** is a copy of **nums**.

Here's a method that shows you how that works. The method **doubleAR()** doubles every element in the array passed to it.

When you pass an individual array element to a method (as opposed to an array variable), a copy of the element is passed to the method and the original value stored in the array is **NOT** modified. It's really important that you understand and stay aware of this distinction.

Slide 10

Modifying Array Elements

Duration: 00:00:55

Advance mode: Auto

Express Yourself

- Exercise 5:** define the method **rotateLeft3()** that takes an array of **int** (length 3) as a parameter. The method "rotates left" the elements in the array so that **{1, 2, 3}** becomes **{2, 3, 1}**. Your method should have no return value, but should just modify the array passed to it. Define the method and then run the JUnit test program, **TestRotateLeft3** to make sure you've done it correctly. Snap a shot of the JUnit window with all the tests showing green.

Notes:

Let's look at a method that modifies the elements in an array. Go ahead and define the method named **rotateLeft3**. It will have one **int** parameter. We're going to assume that the array is always three elements long (although you don't specify that when defining the formal parameter.)

Inside the method, you're going to rotate left each of the elements. If, for instance, you pass it an array containing the values 1, 2, 3, then the 2 moves left into the first position, the 3 moves left into the second position, and the 1 rotates around so that it goes into the last position.

When you define the method, make sure it doesn't have any return type; it will be a void method. As you move the elements around inside the method, you'll actually be modifying the array passed to it. Once you get it to compile,

Slide 11

Express Yourself

Duration: 00:01:20

Advance mode: Auto	run the JUnit test program, TestRotateLeft3, and for Exercise 5, shoot me a screen-shot showing your code and all of the JUnit tests running green. Now that you know how to write access the elements in an array, our next lecture will bring back your old friend the loop to automate processing the individual elements.
--------------------	---